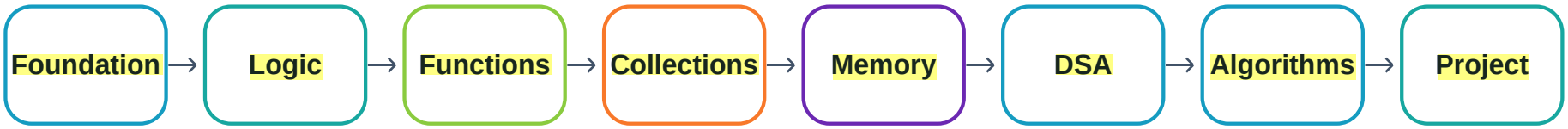
Mentor Coding Guide

36 Sessions: C Foundations to Data Structures, Algorithms, Project and Interview Readiness

C-only learning track aligned to the attached The Talent Grid programming curriculum. The original curriculum spans C, C++ and Python; this guide follows the same learning sequence while implementing the track in C.



Capstone context: Inventory & Employee Records in C, consistent with the curriculum's C project progression from calculator/marksheet to banking with structures, employee records, linked-list contact book and inventory with files.

Candidate outcome: write, dry-run, debug, test, analyze complexity and explain C solutions in an interview-ready way.

36-Session Roadmap

Session	Focus	Phase
1	Programming Environment and How C Runs	Foundations
2	Variables, Data Types, Constants and Casting	Foundations
3	Input, Output and Safe Text Entry	Foundations
4	Operators, Expressions, Precedence and Bitwise Basics	Foundations
5	Decision Making with if, else and Nested Rules	Foundations
6	switch-case, Ternary Decisions and Menu Design	Foundations
7	Loops: for, while, do-while, break and continue	Foundations
8	Nested Loops, Pattern Thinking and Dry Runs	Foundations
9	Functions, Prototypes, Parameters and Return Values	Foundations
10	Scope, Local/Global State and Modular Design	Foundations
11	Recursion and the Call Stack	Foundations
12	One-Dimensional Arrays and Contiguous Storage	Collections & Memory

36-Session Roadmap (continued)

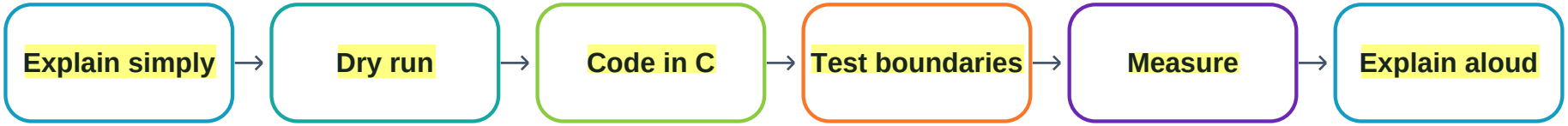
Session	Focus	Phase
13	Array Search, Insert and Delete	Collections & Memory
14	Two-Dimensional Arrays and Matrices	Collections & Memory
15	C Strings and Character Arrays	Collections & Memory
16	String Interview Problems: Palindrome and Anagram	Collections & Memory
17	Pointers: Address, Dereference and Pointer Safety	Collections & Memory
18	Call by Value, Pointer Parameters and Pointer Arithmetic	Collections & Memory
19	Dynamic Memory: malloc, calloc, realloc and free	Collections & Memory
20	Structures and Procedural Data Modeling in C	Collections & Memory
21	File Handling: Persisting Records	Collections & Memory
22	Errors, Debugging, Defensive Programming and C Libraries	Collections & Memory
23	Stack: Last-In, First-Out	DSA & Algorithms
24	Queue: First-In, First-Out and Circular Buffer	DSA & Algorithms

36-Session Roadmap (continued)

Session	Focus	Phase
25	Singly Linked List: Nodes, Insert, Delete and Reverse	DSA & Algorithms
26	Hash Table: Fast Key Lookup in C	DSA & Algorithms
27	Trees and Binary Search Trees	DSA & Algorithms
28	Heap and Priority-Based Retrieval	DSA & Algorithms
29	Graphs, Representations, BFS and DFS	DSA & Algorithms
30	Trie: Prefix Search	DSA & Algorithms
31	Searching, Sorting and Complexity Analysis	DSA & Algorithms
32	Two Pointers and Sliding Window	DSA & Algorithms
33	Recursion Patterns and Backtracking	DSA & Algorithms
34	Greedy Thinking and Bit Manipulation	DSA & Algorithms
35	Dynamic Programming and Core Graph Algorithms	DSA & Algorithms
36	Capstone Integration, Timed Coding and Interview Readiness	Project & Mock

Learning and Validation Model

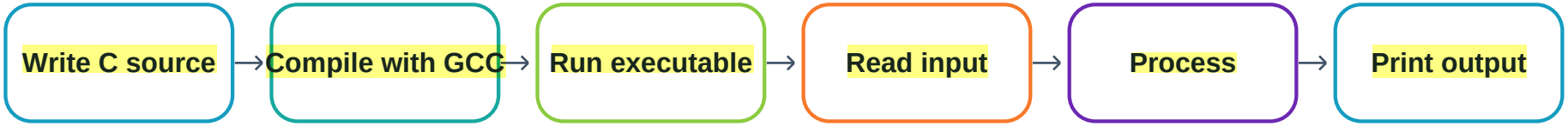
Every session follows the same candidate-facing rhythm: understand the idea, see where it is used, write C code, test edge cases, analyze complexity where relevant, and explain the result aloud.



Area	Weight	Evidence
Syntax & fundamentals	15%	Basic programs without copying
Logic building	20%	Conditions, loops, dry runs
Data structures	20%	Choose and implement core structures
Algorithms	20%	Search, sort, recursion, DP, greedy, graph
Debugging	10%	Boundary, invalid input, memory defects
Code readability	5%	Names, functions, comments, formatting
Project implementation	10%	Working tested C capstone

Session 1: Programming Environment and How C Runs

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.



Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Source code	Human-readable instructions saved in a **.c** file.	It is the maintainable form of the program.	Stores the business rules for inventory, employees and reports.	Editing generated binaries instead of source loses maintainability.	Compile from a clean source file and reproduce the same output.
Compiler	A compiler translates C source into machine code. **GCC** = GNU Compiler Collection.	C must be translated before the CPU can execute it.	Builds the capstone into an executable used by staff.	Warnings ignored during compilation can hide defects.	Compile with **-Wall -Wextra -pedantic** and resolve warnings.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Executable / CPU / memory	Executable = runnable machine-code file. CPU = Central Processing Unit. Memory stores instructions and data while running.	Explains what happens after build and why memory bugs matter.	The executable loads records, processes requests and prints reports.	Invalid memory access can crash or corrupt records.	Run known test cases and use a debugger or memory checker when available.
IDE and terminal	IDE = Integrated Development Environment. Terminal/command line runs compiler and program commands.	Beginners must separate coding from compiling and running.	Developers can build and test the capstone consistently on any machine with GCC.	Depending only on an IDE can hide the real build command.	Run the same program directly from a terminal.

Hands-on practice: Write, compile and run Hello World; then read one integer and echo it. Explain each stage from .c file to output.

Session 1 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```c
#include <stdio.h>
// Import declarations for printf so the program can print output.

int main(void) {
// Define the standard entry point where program execution begins.
    printf("Hello, C!\n");
    // Send a visible message to standard output and end the line.
    return 0;
    // Return zero to tell the operating system that execution
    // succeeded.
}
```

Interview revision: Explain: source -> preprocessing -> compilation -> linking -> executable -> CPU execution.

Session 2: Variables, Data Types, Constants and Casting

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Variable	A named memory location whose value can change.	Programs need named storage for quantities and state.	Stores item quantity, employee ID, price and counters.	Wrong type or uninitialized value can create nonsense results.	Initialize variables and test min/max expected values.
int / float / double / char	**int** stores whole numbers; **float/double** store decimals; **char** stores one character.	Correct types improve accuracy and memory use.	IDs/counts use int; money calculations can use double; codes can use char.	Overflow, rounding or format mismatch can corrupt calculations.	Check ranges, use sizeof, and test boundary values.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
const	**const** marks a value that should not be changed.	Prevents accidental modification of fixed rules.	A maximum record count or tax rate can be declared constant.	Magic numbers scattered across code are hard to maintain.	Change one constant and confirm all dependent logic behaves consistently.
Type casting	Type casting explicitly converts one type to another.	It prevents accidental integer-only arithmetic.	Average stock value may require converting integer totals before division.	Integer division can silently discard the decimal part.	Use known fractions such as 5/2 and confirm 2.5 when intended.

Hands-on practice: Create variables for employee ID, quantity, unit price and grade; calculate stock value and print types logically.

Session 2 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>
// Import standard input/output declarations.

int main(void) {
// Start the program.
    const int max_records = 1000;
    // Store a fixed capacity rule that must not change.
    int quantity = 5;
    // Store a whole-number inventory quantity.
    double unit_price = 249.50;
    // Store a price with fractional precision.
    char grade = 'A';
    // Store one classification character.
    double stock_value = (double)quantity * unit_price;
    // Convert quantity explicitly and calculate total value.
    printf("Max=%d Qty=%d Price=%.2f Grade=%c Value=%.2f\n",
    // Format all values using matching conversion specifiers.
           max_records, quantity, unit_price, grade, stock_value);
        // Supply values in the same order as the format string.
    return 0;
    // Report successful completion.
```

```
}
```

Interview revision: Be ready to explain integer division, overflow, precision and why format specifiers must match data types.

Session 3: Input, Output and Safe Text Entry

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
printf	Formatted output function from **stdio.h**.	Readable output is how users and testers observe program state.	Displays menus, record details, errors and reports.	Wrong format specifier can print garbage or cause undefined behavior.	Compare format specifier with the variable type and expected output.
scanf	Formatted input function for token-based input.	Useful for controlled numeric input.	Reads menu choices, IDs and quantities.	Invalid text input can leave variables unchanged and trap loops.	Check scanf return value equals the number of expected fields.
fgets	Reads a whole line into a bounded character array.	Safer for names and text containing spaces.	Reads employee names and product descriptions.	Buffer size mistakes or leftover newline handling can damage text.	Verify the returned pointer, buffer size and trimmed newline.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Input validation	Rejecting data that violates expected type or range.	Business programs must not accept impossible values.	Rejects negative quantity, blank name or invalid menu choice.	Bad input can poison later calculations and files.	Create invalid, boundary and valid test cases.

Hands-on practice: Build a small form that reads employee ID, name and salary and rejects invalid ID input.

Session 3 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>
// Provide scanf, fgets and printf.
#include <string.h>
// Provide strcspn for safe newline removal.

int main(void) {
// Start the console program.
    int employee_id = 0;
    // Initialize the ID so its state is known.
    char name[80] = {0};
    // Reserve a bounded text buffer and clear it.
    printf("Employee ID: ");
    // Prompt the user for a numeric identifier.
    if (scanf("%d", &employee_id) != 1) {
    // Confirm that exactly one integer was successfully read.
        printf("Invalid ID.\n");
        // Explain the validation failure to the user.
        return 1;
        // Stop with a non-zero status because input was invalid.
    }
    int ch = 0;
```

```
    // Create a character holder used to clear the remaining input line.
    while ((ch = getchar()) != '\n' && ch != EOF) { }
    // Consume leftover characters so fgets starts on a fresh line.
    printf("Employee name: ");
    // Prompt for text that may contain spaces.
    if (fgets(name, sizeof name, stdin) == NULL) {
    // Read at most the buffer capacity and detect input failure.
        printf("Input error.\n");
        // Report that the text could not be read.
        return 1;
        // Stop because a required field is missing.
    }
    name[strcspn(name, "\n")] = '\0';
    // Replace the first newline with the string terminator.
    printf("Saved: %d - %s\n", employee_id, name);
    // Display the validated record for confirmation.
    return 0;
    // Finish successfully.
}
```

Interview revision: Explain when scanf is convenient, when fgets is safer, and why every external input needs validation.

Session 4: Operators, Expressions, Precedence and Bitwise Basics

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Arithmetic operators	+, -, *, / and % perform numeric calculations.	They convert formulas into code.	Calculate invoice totals, stock value and remainder-based rules.	Integer division and divide-by-zero can produce wrong or failed results.	Use hand-calculated examples including zero and negative values.
Relational / logical operators	<, >, <=, >=, ==, != compare values; &&, || and ! combine conditions.	Business rules are built from true/false decisions.	Checks reorder level, salary range and login/menu validity.	Using = instead of == or wrong AND/OR logic changes decisions.	Build truth-table cases for every branch.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Assignment / increment	= stores a value; += and similar update it; ++/-- change by one.	Counters and accumulators depend on predictable updates.	Updates quantities and record counts.	Pre/post increment inside complex expressions can confuse evaluation.	Prefer simple standalone increments and dry-run values.
Bitwise operators	&, |, ^, ~, <<, >> operate on individual bits.	Useful for compact flags and later bit-manipulation interview problems.	Status flags can represent active, verified or locked states.	Confusing logical && with bitwise & changes meaning.	Print binary reasoning for small values and test each flag independently.

Hands-on practice: Create a stock status expression: reorder if quantity is below minimum and item is active.

Session 4 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>
// Provide standard output.

int main(void) {
// Start the program.
    int quantity = 8;
    // Store current stock.
    int minimum = 10;
    // Store the reorder threshold.
    unsigned int flags = 1u;
    // Use bit 0 as an "active" status flag.
    int is_active = (flags & 1u) != 0u;
    // Isolate bit 0 and convert the result into a true/false value.
    int should_reorder = (quantity < minimum) && is_active;
    // Combine a numeric comparison with the status flag.
    printf("Reorder=%s\n", should_reorder ? "yes" : "no");
    // Use the ternary operator only for a small display choice.
    return 0;                                          // Report success.
}
```

Interview revision: Explain precedence safely: use parentheses when the intended grouping is important to a reader.

Session 5: Decision Making with if, else and Nested Rules

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
if	Runs a block only when its condition is true.	Models a single rule.	Detects low stock or invalid salary.	A wrong condition can allow invalid records.	Test one true and one false case.
if-else	Chooses exactly one of two paths.	Needed for pass/fail and accept/reject decisions.	Accepts or rejects a requested stock issue.	Boundary values can be sent to the wrong branch.	Test values just below, at and just above the boundary.
else-if ladder	Checks ordered mutually exclusive ranges.	Useful for grades, categories and thresholds.	Classifies stock health as critical, low, normal or excess.	Poor branch order can make later branches unreachable.	Create one test for every range and inspect branch coverage.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Nested if	An if inside another if for dependent rules.	Some checks only make sense after earlier checks pass.	Only verify available quantity after product ID is valid.	Deep nesting becomes hard to read and easy to miss.	Refactor repeated conditions and measure nesting depth informally.

Hands-on practice: Build stock health classification with explicit boundary tests.

Session 5 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```c
#include <stdio.h>                                  // Provide printf.

int main(void) {                                     // Start the program.
    int quantity = 12;
    // Example current quantity.
    if (quantity < 0) {
    // Reject impossible negative stock first.
        printf("Invalid quantity\n");                // Report invalid data.
    } else if (quantity == 0) {
    // Handle the exact zero boundary.
        printf("Out of stock\n");
        // Report the zero-stock state.
    } else if (quantity < 10) {
    // Handle a low but positive quantity.
        printf("Low stock\n");
        // Report a reorder warning.
    } else {
    // Handle all remaining valid quantities.
        printf("Stock sufficient\n");
        // Report the normal state.
    }
```

```
    return 0;
    // Finish successfully.
}
```

Interview revision: In interviews, say the range of values handled by every branch, not only what the code “looks like”.

Session 6: switch-case, Ternary Decisions and Menu Design

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
switch-case	Selects a branch from discrete integer/character values.	Makes command menus easier to read than long equality ladders.	Routes Add, Search, Update, Delete and Exit commands.	Missing break can unintentionally fall through.	Test every case plus default.
break	Exits the current switch or loop.	Prevents accidental execution of later switch cases.	Stops after one selected menu action.	Missing break may run multiple actions.	Verify only one action occurs per choice unless fall-through is intentional.
default	Fallback case when no listed choice matches.	Provides defensive handling for invalid menu input.	Shows “invalid choice” for unsupported commands.	No default can leave users without feedback.	Pass an out-of-range choice and confirm safe recovery.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Ternary operator	**condition ? a : b** chooses one of two expression values.	Useful for tiny display choices.	Prints active/inactive status.	Nested ternaries quickly become unreadable.	Keep it to short two-way expressions and compare output against if-else.

Hands-on practice: Create a clean menu dispatcher for the capstone.

Session 6 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>
// Provide input and output functions.

int main(void) {                                    // Start the menu program.
    int choice = 0;
    // Store the user's selected menu option.
    printf("1 Add  2 Search  3 Exit\n");
    // Display available commands.
    if (scanf("%d", &choice) != 1) {
    // Validate that the user entered an integer.
        printf("Invalid input\n");                  // Explain the input error.
        return 1;
        // Stop because the menu choice is unusable.
    }
    switch (choice) {
    // Dispatch exactly one discrete menu action.
        case 1:                                     // Handle the Add command.
            printf("Add record\n");
        // Placeholder for the future add function.
            break;
        // Prevent fall-through into the next case.
```

```
        case 2:
        // Handle the Search command.
            printf("Search record\n");
        // Placeholder for the search function.
            break;
        // Prevent fall-through into Exit.
        case 3:                                 // Handle the Exit command.
            printf("Goodbye\n");
        // Confirm normal termination.
            break;
        // End this switch branch explicitly.
        default:
        // Handle all unsupported numeric choices.
            printf("Unknown choice\n");
        // Give safe feedback instead of doing nothing.
    }
    return 0;                                   // Finish the program.
}
```

Interview revision: Explain why switch is suitable for discrete menu choices but not for arbitrary ranges.

Session 7: Loops: for, while, do-while, break and continue

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
for loop	Repeats with initialization, condition and update in one header.	Best when the repetition count is known or index-based.	Traverses arrays and prints reports.	Off-by-one errors can skip or overrun data.	Test first index, last valid index and zero-length logical case.
while loop	Repeats while a condition remains true.	Useful when repetition depends on state rather than a fixed count.	Keeps reading commands until Exit.	If state never changes, the loop becomes infinite.	Prove which statement moves the condition toward false.
do-while	Runs once before checking the condition.	Useful for menus that must display at least once.	Shows the capstone menu and repeats until Exit.	Can run once even when the starting state is invalid.	Test initial invalid and exit choices.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
break / continue	break exits a loop; continue skips to the next iteration.	Controls exceptional cases without duplicating the loop.	Stops a search after a match; skips inactive records.	Overuse can make control flow hard to trace.	Dry-run control movement and ensure required updates still happen.

Hands-on practice: Sum quantities in an array and skip negative values as invalid data.

Session 7 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                  // Provide printf.

int main(void) {
// Start the program.
    int values[] = {5, -1, 7, 3};
    // Create sample stock values including one invalid entry.
    int count = (int)(sizeof values / sizeof values[0]);
    // Calculate how many array elements exist.
    int total = 0;
    // Initialize the accumulator before the loop.
    for (int i = 0; i < count; ++i) {
    // Visit every valid index from zero through count-1.
        if (values[i] < 0) {
        // Detect an invalid negative quantity.
            continue;
        // Skip this record and move to the next iteration.
        }
        total += values[i];
        // Add only validated values into the running total.
    }
    printf("Total valid stock=%d\n", total);
```

```
    // Print the calculated result after traversal.
    return 0;
    // Finish successfully.
}
```

Interview revision: For every loop, state: initialization, continuation condition, update, and termination reason.

Session 8: Nested Loops, Pattern Thinking and Dry Runs

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Nested loop	A loop inside another loop.	Models rows/columns and pairwise relationships.	Processes a matrix of warehouse-by-product quantities.	Wrong row/column bounds can access invalid memory.	Trace outer and inner indexes in a small 2x3 example.
Dry run	Manual execution on paper while tracking variable values.	Builds debugging skill before relying on tools.	Validates report totals and search logic with tiny data.	Skipping dry runs lets logic errors survive despite valid syntax.	Create a trace table with iteration, indexes and changed variables.
Boundary case	Input at an edge such as 0, 1, maximum, empty or last index.	Most loop defects live at boundaries.	Tests no employees, one product and full capacity.	Code may work for middle values but fail at edges.	Maintain a boundary test set for every function.

Hands-on practice: Use a 2D table to compute row totals and grand total.

Session 8 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.

int main(void) {
// Start the matrix example.
    int stock[2][3] = {{2, 4, 6}, {1, 3, 5}};
    // Store two warehouse rows and three product columns.
    int grand_total = 0;
    // Initialize the total across all warehouses.
    for (int row = 0; row < 2; ++row) {
    // Visit each warehouse row.
        int row_total = 0;
        // Reset this warehouse's subtotal.
        for (int col = 0; col < 3; ++col) {
        // Visit each product column in the current row.
            row_total += stock[row][col];
        // Add the current matrix cell to the row subtotal.
        }
        grand_total += row_total;
        // Add the completed row subtotal to the grand total.
        printf("Warehouse %d total=%d\n", row, row_total);
        // Display the subtotal for validation.
```

```
    }
    printf("Grand total=%d\n", grand_total);
    // Display the combined result after both loops finish.
    return 0;
    // Finish successfully.
}
```

Interview revision: Be able to draw a two-column dry-run table showing row/col values and accumulated totals.

Session 9: Functions, Prototypes, Parameters and Return Values

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Function	A named block that performs one focused task.	Improves reuse, testing and readability.	Separate addEmployee, findItem and calculateValue operations.	One giant main function becomes difficult to test and change.	Test each function independently with known inputs.
Prototype	A declaration that tells the compiler a function's name, return type and parameters before use.	Supports modular code and compiler type checking.	Header-like declarations organize capstone operations.	Wrong prototypes can cause warnings or mismatched calls.	Compile with warnings enabled and keep declaration /definition identical.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Parameter / argument	Parameter is a function input variable; argument is the actual value passed.	Allows the same logic to work on different data.	findEmployee accepts the ID being searched.	Passing values in wrong order or type returns wrong results.	Use meaningful names and tests with distinct values.
Return value	A value sent back to the caller.	Lets a function produce a result without printing internally.	Search function returns index; validator returns true/false style integer.	Printing instead of returning makes reuse and testing harder.	Assert expected return value for several cases.

Hands-on practice: Refactor a stock value calculation into a reusable function.

Session 9 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>
// Provide printf.

double stock_value(int quantity, double unit_price);
// Declare the function signature before main uses it.

int main(void) {
// Start the program.
    double value = stock_value(4, 125.50);
    // Call the function with concrete arguments and save its result.
    printf("Value=%.2f\n", value);
    // Print the returned value in the presentation layer.
    return 0;
    // Finish successfully.
}

double stock_value(int quantity, double unit_price) {
// Define the reusable calculation with typed parameters.
    return (double)quantity * unit_price;
    // Compute and return the value without printing side effects.
}
```

Interview revision: Explain why a function should usually do one job and why returning data is more reusable than printing it.

Session 10: Scope, Local/Global State and Modular Design

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Local variable	A variable declared inside a function/block and visible only there.	Limits accidental interference between parts of a program.	A temporary subtotal exists only inside report generation.	Using a local variable after its lifetime ends is invalid.	Check declaration block and avoid returning addresses of local variables.
Global variable	A variable declared outside functions and accessible broadly.	Sometimes useful for true program-wide constants/state but risky.	Could hold a fixed capacity constant, though passing data is usually clearer.	Hidden dependencies make testing difficult.	Count globals and justify each one; prefer parameters for mutable state.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Scope	The region in which a name is visible.	Prevents name collisions and clarifies ownership.	Loop indexes and temporary buffers remain confined to their tasks.	Shadowing can make code update a different variable than intended.	Compile with warnings and use a naming/style review.
Modularity	Splitting the program into cohesive functions/files.	Scales beginner programs into maintainable projects.	Separate input, validation, storage, search and report modules.	Tightly coupled code makes one change break many places.	Measure whether each function can be tested with minimal setup.

Hands-on practice: Create validator and calculator functions with no unnecessary global mutable state.

Session 10 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                  // Provide printf.

int valid_quantity(int quantity) {
// Define a focused validator that depends only on its parameter.
    return quantity >= 0;
    // Return non-zero for valid stock and zero for invalid stock.
}

int main(void) {                                    // Start the program.
    int quantity = 12;
    // Keep business state local to the function that owns it.
    if (!valid_quantity(quantity)) {
    // Call the validator instead of duplicating the rule.
        printf("Invalid quantity\n");
        // Report the failed validation.
        return 1;
        // Stop because the input violates the rule.
    }
    printf("Quantity accepted: %d\n", quantity);
    // Continue only after validation succeeds.
    return 0;                                       // Finish normally.
```

```
}
```

Interview revision: Explain variable lifetime and why minimizing mutable global state improves testability.

Session 11: Recursion and the Call Stack

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Recursion	A function solves a problem by calling itself on a smaller version.	Teaches decomposition and prepares for trees/backtracking.	Can total hierarchical categories or traverse a tree later.	Without a base case, calls continue until stack overflow.	Identify base case, smaller input, and maximum practical depth.
Base case	The condition that stops recursive calls.	Guarantees termination.	Stops a tree traversal at a NULL child.	Missing or unreachable base case causes infinite recursion.	Test the smallest input directly.
Call stack	Memory used to track active function calls, parameters and locals.	Explains recursion memory cost.	Deep hierarchical data creates many active frames.	Excess depth can exhaust stack memory.	Estimate recursion depth and compare with input constraints.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Recursive relation	The rule that connects the current answer to a smaller answer.	Makes recursive correctness explainable.	sum(n) = n + sum(n-1) is a simple learning model.	If the input does not shrink, recursion never reaches the base.	Dry-run 3-4 levels and show the return path.

Hands-on practice: Implement factorial with input validation and explain stack frames.

Session 11 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                  // Provide printf.

unsigned long long factorial(unsigned int n) {
// Define a recursive function for a non-negative integer.
    if (n <= 1u) {
    // Check the base case that terminates recursion.
        return 1u;
        // Return the known factorial for 0 or 1.
    }
    return (unsigned long long)n * factorial(n - 1u);
    // Multiply n by the solution of the smaller subproblem.
}

int main(void) {
// Start the test program.
    unsigned int n = 5u;
    // Choose a small value whose result is easy to verify manually.
    printf("%u! = %llu\n", n, factorial(n));
    // Call the recursive function and print its result.
    return 0;
    // Finish successfully.
```

```
}
```

Interview revision: Say both time O(n) and auxiliary stack space O(n) for this factorial implementation.

Session 12: One-Dimensional Arrays and Contiguous Storage

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Array	A fixed-size sequence of same-type elements stored contiguously.	Provides O(1) indexed access and is the base for many data structures.	Stores a fixed batch of quantities or employee IDs.	Index outside 0..n-1 causes undefined behavior.	Check every index against logical length and capacity.
Indexing	Accessing an element by position, starting at index 0.	Makes direct reads/updates fast.	Update quantity for product at known index.	Off-by-one mistakes read/write the wrong memory.	Test index 0 and index n-1; reject n.
Traversal	Visiting each element, usually with a loop.	Needed for totals, searches and reports.	Sum stock or find highest salary.	Wrong loop bound skips or overruns elements.	Compare traversed count with logical length.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Capacity vs length	Capacity is allocated slots; length is currently used slots.	Essential for manual insertion /deletion in C arrays.	Capstone may allocate 1000 employee slots but use only 120.	Treating capacity as length may process garbage values.	Maintain an explicit count and assert 0 <= count <= capacity.

Hands-on practice: Calculate min, max and average from an integer array.

Session 12 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>
// Provide printf.

int main(void) {
// Start the array analysis.
    int values[] = {12, 7, 19, 10};
    // Create a fixed contiguous collection of quantities.
    int n = (int)(sizeof values / sizeof values[0]);
    // Derive the number of elements from total bytes and element bytes.
    int minimum = values[0];
    // Initialize minimum from the first valid element.
    int maximum = values[0];
    // Initialize maximum from the first valid element.
    int total = 0;
    // Initialize the running sum.
    for (int i = 0; i < n; ++i) {
    // Visit every valid index exactly once.
        if (values[i] < minimum) minimum = values[i];
        // Update minimum when a smaller value is found.
        if (values[i] > maximum) maximum = values[i];
        // Update maximum when a larger value is found.
```

```
        total += values[i];
        // Add the current value to the sum.
    }
    double average = (double)total / n;
    // Cast before division so the fractional average is preserved.
    printf("min=%d max=%d avg=%.2f\n", minimum, maximum, average);
    // Print the verified summary.
    return 0;
    // Finish successfully.
}
```

Interview revision: Explain why array access is O(1) but full traversal is O(n).

Session 13: Array Search, Insert and Delete

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Linear search	Checks elements one by one until a match is found or data ends.	Works even when the array is unsorted.	Find an employee ID in a small unsorted record list.	Stopping early incorrectly may miss a later match.	Test first, middle, last and absent targets.
Insertion	Adds an element while preserving logical order by shifting elements.	Shows why arrays have expensive middle updates.	Insert a record into a sorted ID list.	No capacity check can write beyond allocated storage.	Confirm capacity first and verify shifted region.
Deletion	Removes an element and closes the gap by shifting left.	Maintains a compact logical array.	Delete an inactive record from an in-memory list.	Wrong shift bounds duplicate or lose values.	Compare before/after length and exact sequence.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Time trade-off	Array indexing is fast, but middle insert/delete can be O(n).	Prepares students to choose linked lists later.	High-update datasets may need a different structure.	Choosing by habit can hurt performance.	Measure number of moved elements for worst-case in sertion/deletion.

Hands-on practice: Implement a safe linear search returning the index or -1.

Session 13 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.

int find_index(const int values[], int n, int target) {
// Accept read-only array data, length and search key.
    for (int i = 0; i < n; ++i) {
    // Scan from the first valid index to the last.
        if (values[i] == target) {
        // Compare the current element with the requested value.
            return i;
        // Return immediately when the first match is found.
        }
    }
    return -1;
    // Use -1 as a clear "not found" result.
}

int main(void) {                                        // Start the test.
    int ids[] = {104, 101, 110, 108};
    // Create an intentionally unsorted collection.
    int n = (int)(sizeof ids / sizeof ids[0]);
    // Compute the element count.
```

```
    int index = find_index(ids, n, 110);
    // Search without assuming sorted order.
    printf("index=%d\n", index);
    // Print the returned index for validation.
    return 0;
    // Finish successfully.
}
```

Interview revision: State linear search time O(n), space O(1), and why sortedness is not required.

Session 14: Two-Dimensional Arrays and Matrices

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
2D array	An array whose elements are themselves fixed-size arrays.	Models row-column data with predictable memory layout.	Warehouse x product stock table.	Swapping row and column limits can overrun memory.	Label dimensions and test final row/column indexes.
Row-major order	C stores rows contiguously one row after another.	Explains memory locality and pointer calculations.	Sequential row scans are cache-friendly for reports.	Assuming a different layout gives wrong address arithmetic.	Print addresses of neighboring elements as a learning check.
Matrix traversal	Nested loops visit every row and column.	Required for totals, transpose and search.	Build per-warehouse and per-product totals.	Incorrect nested bounds corrupt results.	Compare processed cell count with rows * columns.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Transpose concept	Rows become columns and columns become rows.	Common matrix interview exercise.	Transforms reports from warehouse-centric to product-centric view.	Writing in-place without square-matrix care can overwrite data.	Use separate output matrix first and compare dimensions.

Hands-on practice: Compute column totals for product quantities across warehouses.

Session 14 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.

int main(void) {
// Start the matrix example.
    int stock[2][3] = {{2, 4, 6}, {1, 3, 5}};
    // Store 2 warehouses by 3 products.
    for (int col = 0; col < 3; ++col) {
    // Choose one product column at a time.
        int total = 0;
        // Reset this product's total before scanning warehouses.
        for (int row = 0; row < 2; ++row) {
        // Visit every warehouse for the current product.
            total += stock[row][col];
        // Add this warehouse's quantity into the product total.
        }
        printf("Product %d total=%d\n", col, total);
        // Print the completed column aggregation.
    }
    return 0;
    // Finish successfully.
}
```

Interview revision: Be able to calculate total elements and explain why the second dimension matters in function parameters.

Session 15: C Strings and Character Arrays

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
C string	A char array terminated by the null character **\0**.	Text handling in C depends on correct termination and capacity.	Stores product names, employee names and descriptions.	Missing terminator can make functions read beyond the buffer.	Ensure buffer size includes space for \0 and inspect length.
strlen	Returns the number of characters before \0.	Supports validation and traversal.	Checks that names are non-empty and within policy.	Calling it on non-terminated data is unsafe.	Compare with known literal lengths.
strcmp	Lexicographically compares two strings.	C strings cannot be compared with == for contents.	Matches a product code or employee name.	Using == compares addresses, not text content.	Test equal, less-than and greater-than examples.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
string.h	Standard header containing common string functions.	Avoids reimplementing reliable primitives.	Used for lengths, c opies/comparisons when bounds are controlled.	Unbounded copying can overflow destination buffers.	Track destination capacity and prefer validated input lengths.

Hands-on practice: Read a line safely, trim newline and compare it with a target name.

Session 15 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>
// Provide fgets and printf.
#include <string.h>
// Provide strcspn and strcmp.

int main(void) {
// Start the string example.
    char name[40] = {0};
    // Reserve bounded storage and initialize it.
    printf("Product name: ");
    // Ask the user for a full line of text.
    if (fgets(name, sizeof name, stdin) == NULL) {
    // Read safely within the array capacity.
        return 1;
        // Stop if input could not be obtained.
    }
    name[strcspn(name, "\n")] = '\0';
    // Remove the trailing newline while preserving termination.
    if (strcmp(name, "Keyboard") == 0) {
    // Compare string contents rather than array addresses.
        printf("Matched product\n");
```

```
        // Confirm the expected text match.
    } else {
    // Handle all other valid names.
        printf("Different product\n");           // Report a non-match.
    }
    return 0;
    // Finish successfully.
}
```

Interview revision: Explain why char name[40] is storage, while a C string is the characters up to the first null terminator.

Session 16: String Interview Problems: Palindrome and Anagram

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Palindrome	Text that reads the same forward and backward.	Builds index, two-pointer and string reasoning.	Can validate symmetric codes or simply serve as interview practice.	Ignoring case/spaces can change the intended definition.	State normalization rules before testing.
Anagram	Two strings made from the same character counts in a different order.	Introduces frequency counting and hashing ideas.	Can compare normalized labels or serve as interview practice.	Different alphabets/case assumptions can break counting.	Verify equal normalized lengths and identical frequency tables.
Two-pointer scan	Uses one index from each end moving inward.	Reduces palindrome checking to O(n) time and O(1) extra space.	Efficiently validates a code without creating a reversed copy.	Incorrect pointer movement can skip or repeat comparisons.	Trace left/right positions until they cross.

Hands-on practice: Implement a case-sensitive palindrome check for a null-terminated string.

Session 16 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.
#include <string.h>                                     // Provide strlen.

int is_palindrome(const char text[]) {
// Accept a read-only C string.
    size_t left = 0;
    // Start the left pointer at the first character.
    size_t right = strlen(text);
    // Compute the logical string length.
    if (right == 0) return 1;
    // Treat the empty string as a palindrome by this definition.
    --right;
    // Convert length into the last valid character index.
    while (left < right) {
    // Continue while there are unmatched pairs.
        if (text[left] != text[right]) return 0;
        // Reject immediately when a mirrored pair differs.
        ++left;
        // Move the left pointer inward.
        --right;
        // Move the right pointer inward.
```

```
    }
    return 1;
    // Accept when all mirrored pairs matched.
}

int main(void) {
// Start the test program.
    printf("%d\n", is_palindrome("level"));
    // Validate the function with a known palindrome.
    return 0;
    // Finish successfully.
}
```

Interview revision: State assumptions first: case sensitivity, spaces/punctuation, character set, and whether empty text counts.

Session 17: Pointers: Address, Dereference and Pointer Safety

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Address	A location in memory; **&** obtains the address of an object.	Pointers are central to C memory, arrays and data structures.	Functions can receive addresses of records to update them.	Using an invalid address can crash or corrupt data.	Only dereference pointers known to point to live compatible objects.
Pointer	A variable that stores an address.	Enables indirect access and dynamic structures.	Linked-list nodes connect through pointers.	Uninitialized, dangling or NULL pointers are dangerous.	Initialize pointers, check NULL and define ownership.
Dereference	***pointer** accesses the object at the stored address.	Allows reading or changing the pointed value.	Update a quantity through a pointer parameter.	Dereferencing NULL or stale memory is undefined behavior.	Check pointer validity before dereference.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
NULL	A special pointer value meaning “points to no object”.	Provides a clear empty/not-found state.	End of linked list or failed allocation.	Treating NULL as a real object causes a crash.	Guard every nullable pointer before use.

Hands-on practice: Change a variable through a pointer and print both address and value.

Session 17 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.

int main(void) {
// Start the pointer example.
    int quantity = 10;
    // Create a normal integer object in memory.
    int *ptr = &quantity;
    // Store the address of quantity in a compatible pointer.
    printf("value=%d address=%p\n", quantity, (void*)ptr);
    // Print the value and a portable pointer representation.
    *ptr = 15;
    // Dereference the pointer and update the original object.
    printf("updated=%d\n", quantity);
    // Confirm that indirect mutation changed the same variable.
    ptr = NULL;
    // Clear the pointer when it no longer needs to reference the
    // object.
    return 0;
    // Finish successfully.
}
```

Interview revision: Explain the difference between pointer value (address) and pointed-to value (object contents).

Session 18: Call by Value, Pointer Parameters and Pointer Arithmetic

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Call by value	C passes function arguments by value: the parameter receives a copy.	Explains why ordinary parameters cannot directly replace caller variables.	A calculation function can safely receive copied numbers.	Expecting a copied parameter to modify the caller gives surprising results.	Change the parameter inside a function and observe caller value unchanged.
Pointer parameter	A function parameter can receive an address, allowing mutation of the pointed object.	Common C pattern for outputs and record updates.	updateQuantity receives a pointer to an item record/field.	Wrong or NULL pointer can corrupt memory.	Check pointer and use clear o wnership/mutati on documentation.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Pointer arithmetic	Adding 1 to T* moves by sizeof(T) bytes to the next element.	Explains array-pointer relationship and traversal.	Iterate through contiguous records.	Moving outside the array object is invalid.	Keep pointer range within [begin, one-past-end].
Array decay	In most expressions, an array name converts to a pointer to its first element.	Explains function parameters like int a[].	Passes an array efficiently to search/sort functions.	sizeof on a function parameter gives pointer size, not array length.	Pass length explicitly and test it.

Hands-on practice: Use a pointer parameter to swap two integers.

Session 18 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

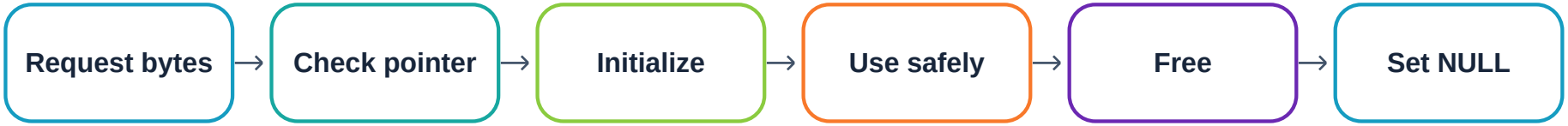
```
#include <stdio.h>                                      // Provide printf.

void swap_int(int *a, int *b) {
// Receive addresses so the function can mutate caller variables.
    if (a == NULL || b == NULL) return;
    // Defensively stop when either pointer is unusable.
    int temp = *a;
    // Copy the first pointed value into temporary storage.
    *a = *b;
    // Write the second pointed value into the first object.
    *b = temp;
    // Write the saved first value into the second object.
}

int main(void) {
// Start the test.
    int x = 3;
    // Create the first integer.
    int y = 9;
    // Create the second integer.
    swap_int(&x, &y);
```

```
    // Pass addresses so swap_int can change both originals.
    printf("x=%d y=%d\n", x, y);
    // Verify that the caller variables were exchanged.
    return 0;
    // Finish successfully.
}
```

Interview revision: Phrase it accurately: C is call-by-value; passing a pointer is passing an address value that enables indirect mutation.

Session 19: Dynamic Memory: malloc, calloc, realloc and free

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.



Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Heap / dynamic memory	Memory obtained during runtime and released manually.	Allows sizes not known at compile time.	Loads a variable number of inventory records.	Leaks, double free and use-after-free can destabilize the program.	Track allocation ownership; use a memory checker when available.
malloc	Allocates a requested number of bytes; contents are uninitialized.	Provides raw dynamic storage.	Create n employee records based on file/input count.	Using memory before initialization produces unpredictable data.	Check returned pointer for NULL and initialize elements.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
calloc	Allocates array storage and zero-initializes bytes.	Convenient for zero-based counters/arrays.	Create a zeroed frequency table or record array.	Assuming all semantic defaults equal zero can still be wrong.	Check allocation and explicit business defaults.
realloc / free	realloc resizes an allocation; free releases it.	Supports growing buffers and prevents leaks.	Grow record storage and release it on exit.	Assigning realloc directly can lose original pointer on failure; double free is invalid.	Use temporary pointer for realloc and set freed pointer to NULL.

Hands-on practice: Dynamically allocate an integer array, initialize it and free it.

Session 19 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.
#include <stdlib.h>
// Provide malloc and free.

int main(void) {
// Start the dynamic-memory example.
    size_t n = 5;
    // Decide the runtime element count.
    int *values = malloc(n * sizeof *values);
    // Allocate enough bytes for n integers without duplicating the type
    // name.
    if (values == NULL) {
    // Detect allocation failure before dereferencing.
        printf("Allocation failed\n");
        // Report the resource failure.
        return 1;
        // Stop safely because storage is unavailable.
    }
    for (size_t i = 0; i < n; ++i) {
    // Visit each allocated element within bounds.
        values[i] = (int)(i * 10);
```

```
        // Initialize every element before it is read.
    }
    printf("last=%d\n", values[n - 1]);
    // Use a valid element while the allocation is still alive.
    free(values);
    // Release heap memory exactly once when ownership ends.
    values = NULL;
    // Remove the stale address to reduce accidental reuse.
    return 0;
    // Finish successfully.
}
```

Interview revision: Explain the lifecycle: allocate -> validate -> initialize -> use within bounds -> free once -> stop using the pointer.

Session 20: Structures and Procedural Data Modeling in C

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
struct	A user-defined type that groups related fields.	Lets C model real records even without classes.	Represents Product or Employee with ID, name and value.	Unrelated parallel arrays can get out of sync.	Create sample records and verify every field stays associated.
Member access	Use . for a struct object and -> for a pointer to struct.	Required to read/update record fields clearly.	product.quantity or productPtr->quantity.	Mixing . and -> causes compile errors or wrong reasoning.	Compile warnings/errors and unit tests on field updates.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Procedural encapsulation	C has no class/inheritance language features; data can be grouped in structs and manipulated through focused functions.	Keeps the C-only track honest while still organizing behavior.	Functions such as product_set_quantity enforce rules around Product.	Direct uncontrolled field changes can bypass validation.	Route sensitive updates through validation functions and test rejected states.
Record array	An array of structs stores many same-shaped entities.	Forms the base of in-memory employee/inventory tables.	Search, sort and persist records.	Capacity and duplicate-ID errors can create inconsistent data.	Enforce unique IDs and logical length <= capacity.

Hands-on practice: Define Product and a safe update function.

Session 20 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.

typedef struct {
// Define a reusable record shape for one product.
    int id;
    // Store the unique product identifier.
    char name[40];
    // Reserve bounded text storage for the product name.
    int quantity;
    // Store current stock quantity.
    double price;
    // Store unit price with decimal precision.
} Product;
// Create the convenient type name Product.

int set_quantity(Product *product, int quantity) {
// Accept a record address so the function can update it.
    if (product == NULL || quantity < 0) return 0;
    // Reject missing records and impossible negative stock.
    product->quantity = quantity;
    // Update the field only after validation passes.
```

```
    return 1;
    // Report that the update succeeded.
}

int main(void) {
// Start the record example.
    Product p = {101, "Keyboard", 5, 799.0};
    // Initialize all fields of one product in a clear order.
    set_quantity(&p, 8);
    // Update the product through the validation function.
    printf("%d %s %d %.2f\n", p.id, p.name, p.quantity, p.price);
    // Display the complete record.
    return 0;
    // Finish successfully.
}
```

Interview revision: Explain that C structs group data; C itself does not provide classes, constructors, inheritance or virtual polymorphism.

Session 21: File Handling: Persisting Records

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
FILE*	A handle used by stdio functions to access an open file stream.	Connects in-memory data to persistent storage.	Saves employee/inventory records across program runs.	Using a NULL file handle after fopen failure is unsafe.	Check fopen result every time.
fopen modes	Modes such as r, w and a control reading, replacement writing and appending.	Wrong mode can destroy or fail to create data.	Append audit logs; read records; rewrite updated data.	Opening with w truncates an existing file.	Use temporary test files and verify size/content before production-like use.
fprintf / fscanf / fgets	Write/read formatted text or whole lines.	Supports simple CSV-like and log formats.	Persists record fields and reads them later.	Unescaped delimiters and malformed lines can break parsing.	Round-trip: write known records, read them back, compare fields.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
fclose / file errors	Closing flushes resources; I/O operations can fail.	Data integrity depends on detected errors and cleanup.	Ensures saved inventory changes are actually written.	Ignoring return/errors can silently lose updates.	Check write counts/status and close results where relevant.

Hands-on practice: Append a simple inventory event to a log file.

Session 21 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>
// Provide FILE, fopen, fprintf and fclose.

int main(void) {
// Start the file example.
    FILE *file = fopen("inventory.log", "a");
    // Open the log in append mode so earlier history is preserved.
    if (file == NULL) {
    // Detect failure such as permission or path problems.
        perror("inventory.log");
        // Print the operating-system reason for the failure.
        return 1;
        // Stop because the requested persistence operation cannot
        // proceed.
    }
    if (fprintf(file, "ID=%d QTY=%d\n", 101, 8) < 0) {
    // Write one structured line and detect output failure.
        fclose(file);
        // Release the file handle even on the error path.
        return 1;
        // Report unsuccessful execution.
```

```
    }
    if (fclose(file) != 0) {
    // Close the stream and detect a close/flush failure.
        return 1;
        // Report the persistence failure to the caller/OS.
    }
    return 0;
    // Finish successfully after the record is persisted.
}
```

Interview revision: Know the destructive difference between "w" and "a", and always describe your file format and failure handling.

Session 22: Errors, Debugging, Defensive Programming and C Libraries

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Syntax / compile error	Code violates C grammar or type rules and cannot build correctly.	Fast feedback prevents execution of malformed code.	Compiler catches missing semicolons or wrong declarations.	Ignoring warnings can leave subtle defects.	Use strict warnings and fix them before testing.
Runtime error	Program builds but fails while running.	Memory, files and arithmetic can fail only at runtime.	NULL pointer, divide-by-zero or failed file access.	May crash or corrupt data.	Reproduce with smallest failing test and inspect state.
Logical error	Program runs but produces wrong results.	Most interview and business defects are logical.	Wrong reorder threshold or incorrect search.	Can silently produce believable but wrong output.	Use hand-computed expected results and boundary tests.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Defensive programming	Validate assumptions before using data/resources.	Prevents invalid input from spreading.	Check pointers, counts, ranges and file handles.	Overtrusting input causes undefined or inconsistent behavior.	Measure test coverage across valid, invalid and boundary classes.
Standard libraries	Headers such as stdio.h, stdlib.h, string.h and math.h expose reusable functions.	Reduces reinvention and improves clarity.	I/O, allocation, strings and calculations in the capstone.	Wrong declarations or unchecked library results can still fail.	Read function contracts; check return values and preconditions.

Hands-on practice: Debug a validator by deliberately testing invalid, boundary and normal values.

Session 22 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.

int valid_discount(double percent) {
// Encapsulate the allowed discount rule.
    return percent >= 0.0 && percent <= 100.0;
    // Accept the complete inclusive business range.
}

int main(void) {
// Start the defensive test harness.
    double tests[] = {-1.0, 0.0, 50.0, 100.0, 101.0};
    // Include invalid-low, both boundaries, normal and invalid-high
    // cases.
    int n = (int)(sizeof tests / sizeof tests[0]);
    // Calculate how many test values will be executed.
    for (int i = 0; i < n; ++i) {
    // Run the same validator against each test class.
        printf("%.1f -> %s\n", tests[i], valid_discount(tests[i]) ? "valid" : "inv
        // Display actual behavior for comparison.
    }
    return 0;
```

```
    // Finish after all tests are run.
}
```

Interview revision: When debugging, classify the defect first: compile-time, runtime, or logical; then minimize the failing input.

Session 23: Stack: Last-In, First-Out

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Stack	A data structure where the last pushed item is the first popped. **LIFO** = Last In, First Out.	Matches nested/undo behavior and call-stack reasoning.	Supports undo history or expression processing in a C tool.	Pop/peek on an empty stack is invalid; fixed stack can overflow.	Track size and test empty, one element and full capacity.
push	Adds an item to the top.	Core stack update.	Adds latest operation to undo history.	No capacity check writes past array.	Confirm size increments once and top matches pushed value.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
pop	Removes and returns the top item.	Processes most recent item first.	Undo latest action.	Underflow when empty.	Confirm size decrements and next top is correct.
top/peek	Reads the top without removal.	Allows inspection while preserving state.	Preview latest pending action.	Assuming a top exists on empty stack.	Return status plus output value or check empty first.

Hands-on practice: Implement a fixed-capacity integer stack with guarded push/pop.

Session 23 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.

#define CAPACITY 5
// Define the fixed stack capacity in one place.
typedef struct { int data[CAPACITY]; int size; } Stack;
// Group stack storage and logical size.

int push(Stack *s, int value) {
// Add one value when space is available.
    if (s == NULL || s->size >= CAPACITY) return 0;
    // Reject missing stack or overflow.
    s->data[s->size] = value;
    // Write into the first unused array slot.
    s->size++;
    // Increase logical size after successful write.
    return 1;                                             // Report success.
}

int pop(Stack *s, int *out) {
// Remove the most recently pushed value.
    if (s == NULL || out == NULL || s->size == 0) return 0;
```

```
    // Reject invalid pointers or underflow.
    s->size--;
    // Move size back to the index of the old top.
    *out = s->data[s->size];
    // Return the old top value through the output pointer.
    return 1;                                       // Report success.
}

int main(void) {
// Start a minimal stack test.
    Stack s = {{0}, 0};
    // Initialize storage and set the stack to empty.
    int value = 0;
    // Reserve output storage for pop.
    push(&s, 10);
    // Push the first item.
    push(&s, 20);
    // Push a newer item above it.
    pop(&s, &value);
    // Remove the newest item according to LIFO order.
    printf("popped=%d\n", value);
    // Verify that 20 was returned first.
    return 0;
    // Finish successfully.
}
```

Interview revision: State O(1) push/pop for an array-backed fixed stack and identify overflow/underflow explicitly.

Session 24: Queue: First-In, First-Out and Circular Buffer

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Queue	A structure where the first inserted item is the first removed. **FIFO** = First In, First Out.	Models fair arrival-order processing.	Processes employee requests or inventory tasks in arrival order.	Naive array shifting makes every dequeue O(n).	Use front/rear indexes and verify sequence order.
enqueue	Adds an item at the rear.	Extends pending work.	Adds a new restock request.	Full queue can overwrite live data without checks.	Test capacity and rear wrap-around.
dequeue	Removes an item from the front.	Processes oldest work first.	Handles earliest request first.	Empty queue underflow.	Test empty, one item and multiple items.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Circular queue	Array indexes wrap to reuse freed slots.	Keeps enqueue/ dequeue O(1) with fixed storage.	Efficient bounded task queue.	Incorrect wrap logic confuses full and empty states.	Maintain explicit size and test wrap across array end.

Hands-on practice: Implement a compact circular queue skeleton.

Session 24 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.

#define QCAP 4
// Define queue capacity.
typedef struct { int data[QCAP]; int front; int size; } Queue;
// Store bounded data, front index and logical size.

int enqueue(Queue *q, int value) {
// Add one value at the logical rear.
    if (q == NULL || q->size >= QCAP) return 0;
    // Reject invalid queue or overflow.
    int rear = (q->front + q->size) % QCAP;
    // Compute the wrapped index of the next free slot.
    q->data[rear] = value;
    // Store the new value at that rear slot.
    q->size++;
    // Increase the number of live queue elements.
    return 1;
    // Report success.
}
int dequeue(Queue *q, int *out) {
```

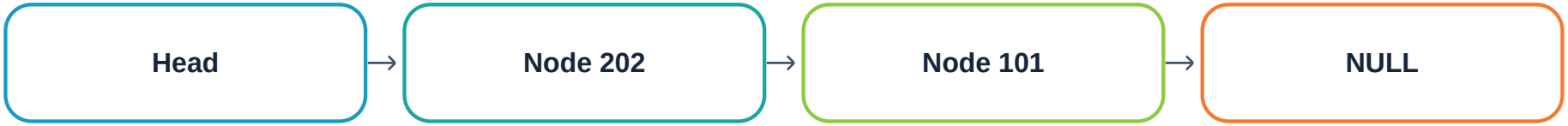
```
// Remove the oldest queued value.
    if (q == NULL || out == NULL || q->size == 0) return 0;
    // Reject invalid pointers or underflow.
    *out = q->data[q->front];
    // Copy the oldest value to caller output storage.
    q->front = (q->front + 1) % QCAP;
    // Advance front and wrap when it reaches capacity.
    q->size--;
    // Decrease logical size after removal.
    return 1;
    // Report success.
}

int main(void) {
// Start a basic FIFO test.
    Queue q = {{0}, 0, 0};
    // Initialize an empty queue.
    int x = 0;
    // Reserve output storage.
    enqueue(&q, 11);
    // Add the first request.
    enqueue(&q, 22);
    // Add the second request.
    dequeue(&q, &x);
    // Remove the oldest request first.
```

```
    printf("dequeued=%d\n", x);
    // Verify FIFO behavior returns 11.
    return 0;
    // Finish successfully.
}
```

Interview revision: Compare stack LIFO with queue FIFO and explain why both can provide O(1) core operations with suitable representation.

Session 25: Singly Linked List: Nodes, Insert, Delete and Reverse

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.



Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Linked list	A sequence of nodes connected by pointers instead of contiguous slots.	Supports flexible growth and O(1) insertion after a known node.	Advanced curriculum project: linked-list contact book or dynamic inventory records.	Lost links cause memory leaks or unreachable data.	Draw pointer changes before coding and count reachable nodes after each operation.
Node	A struct containing data plus pointer(s) to other nodes.	Combines payload and linkage.	Each contact/product record becomes one node.	Uninitialized next pointer can wander into invalid memory.	Initialize next to NULL or a known node.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Head	Pointer to the first node or NULL for empty list.	Provides entry to the entire structure.	Capstone list starts from head and traverses records.	Losing head loses access to the list.	Treat head updates carefully and test empty-list operations.
Insert/delete/reverse	Pointer rewiring changes sequence without shifting an array.	Core linked-list interview operations.	Add/remove contacts and reverse display order.	Changing pointers in wrong order can lose the rest of the list.	Use small 0/1/3-node diagrams and memory cleanup tests.

Hands-on practice: Insert at front and free the entire list.

Session 25 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.
#include <stdlib.h>
// Provide malloc and free.

typedef struct Node {
// Define one linked-list node.
    int id;
    // Store the node's record identifier.
    struct Node *next;
    // Store the address of the next node or NULL.
} Node;
// Create the convenient Node type.

int push_front(Node **head, int id) {
// Receive the address of head so the function can replace it.
    Node *node = malloc(sizeof *node);
    // Allocate one node dynamically.
    if (node == NULL) return 0;
    // Report failure if memory could not be obtained.
    node->id = id;
    // Initialize the payload before linking the node.
```

```
    node->next = *head;
    // Point the new node at the previous first node.
    *head = node;
    // Make the new node the list head.
    return 1;
    // Report successful insertion.
}

void free_list(Node **head) {
// Release every node and clear caller head.
    while (head != NULL && *head != NULL) {
    // Continue while a live node remains.
        Node *old = *head;
        // Save the current first node before changing head.
        *head = old->next;
        // Advance head so the rest of the list remains reachable.
        free(old);
        // Release the detached node exactly once.
    }
}

int main(void) {
// Start the list test.
    Node *head = NULL;
    // Represent an initially empty list.
```

```
    push_front(&head, 101);
    // Insert the first node.
    push_front(&head, 202);
    // Insert another node before it.
    printf("head=%d\n", head->id);
    // Verify that the latest front insertion is now first.
    free_list(&head);
    // Release all heap nodes before program exit.
    return 0;
    // Finish successfully.
}
```

Interview revision: For pointer-changing questions, narrate the links before and after each assignment; never code the rewiring blindly.

Session 26: Hash Table: Fast Key Lookup in C

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Hash table	Stores key-associated data in buckets chosen by a hash function.	Provides average O(1) insert/find when well designed.	Quickly locate employee/product records by integer ID.	Poor hashing or high load creates long collision chains.	Measure load factor, collision count and lookup behavior.
Hash function	Maps a key to a bucket index.	Determines distribution quality.	id % bucket_count is a simple learning hash for integer IDs.	Patterns in keys can cluster badly.	Test bucket distribution across realistic sample IDs.
Collision	Two keys map to the same bucket.	Collisions are normal and must be handled correctly.	Separate chaining can store multiple records in one bucket list.	Overwriting on collision loses records.	Insert two colliding keys and verify both remain findable.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Load factor	items / buckets, often written with Greek alpha conceptually.	Predicts collision pressure and performance.	Signals when a fixed learning table is becoming crowded.	Very high load degrades lookup toward O(n).	Track item count / bucket count and collision lengths.

Hands-on practice: Implement a tiny separate-chaining hash table for integer IDs.

Session 26 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.
#include <stdlib.h>
// Provide malloc and free.

#define BUCKETS 5
// Keep a small fixed bucket count for the learning example.
typedef struct Entry { int key; struct Entry *next; } Entry;
// Store one key plus the next collision-chain entry.

size_t hash_key(int key) {
// Convert an integer key into a valid bucket index.
    return (size_t)((key < 0 ? -key : key) % BUCKETS);
    // Normalize sign and reduce into 0..BUCKETS-1.
}

int insert(Entry *table[], int key) {
// Add a key at the front of its collision chain.
    size_t bucket = hash_key(key);
    // Choose the destination bucket deterministically.
    Entry *entry = malloc(sizeof *entry);
    // Allocate one chain node.
```

```
    if (entry == NULL) return 0;
    // Stop safely if allocation fails.
    entry->key = key;
    // Store the requested key.
    entry->next = table[bucket];
    // Preserve the previous chain by linking it after the new entry.
    table[bucket] = entry;
    // Make the new entry the bucket head.
    return 1;
    // Report successful insertion.
}

int contains(Entry *table[], int key) {
// Search only the bucket selected by the same hash function.
    for (Entry *p = table[hash_key(key)]; p != NULL; p = p->next) {
    // Traverse the collision chain.
        if (p->key == key) return 1;
        // Report success immediately when the key matches.
    }
    return 0;
    // Report that the key is absent after the chain ends.
}

int main(void) {
// Start a focused hash-table test.
```

```
    Entry *table[BUCKETS] = {NULL};
    // Initialize every bucket as an empty chain.
    insert(table, 10);                              // Insert a key.
    insert(table, 15);
    // Insert a colliding key because both map to bucket 0.
    printf("15=%d\n", contains(table, 15));
    // Verify the colliding key remains findable.
    return 0;
    // End the minimal demonstration; production code would also free
    // all entries.
}
```

Interview revision: Explain average O(1) lookup versus worst-case O(n), and always mention the collision strategy.

Session 27: Trees and Binary Search Trees

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Tree	A hierarchical node structure with parent-child relationships.	Models hierarchy and supports recursive traversal.	Could organize categories or serve as interview-grade hierarchical storage.	Cycles or broken links violate tree assumptions.	Traverse and count nodes; verify structure invariants.
Binary tree	Each node has at most left and right children.	Foundation for recursive tree problems.	Represents a hierarchical classification.	Confusing binary tree with BST leads to incorrect search assumptions.	State whether an ordering rule exists.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
BST	Binary Search Tree: keys in left subtree are smaller and keys in right subtree are larger under a chosen duplicate policy.	Enables ordered search/insertion with average O(log n) when reasonably balanced.	Can index unique product IDs for learning.	A sorted insertion sequence can degenerate to a linked list and O(n).	Check inorder traversal is sorted and measure height.
Traversal	Preorder, inorder and postorder visit nodes in different orders.	Required for searching, display and cleanup.	Inorder prints BST IDs in sorted order; postorder is natural for freeing.	Wrong base case or child order changes result.	Compare traversal sequence against a hand-drawn tree.

Hands-on practice: Insert integers into a BST and print inorder traversal.

Session 27 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.
#include <stdlib.h>                                     // Provide malloc.

typedef struct TNode { int key; struct TNode *left, *right; } TNode;
// Define one binary-tree node.

TNode *insert(TNode *root, int key) {
// Insert a key and return the possibly new subtree root.
    if (root == NULL) {
    // A NULL position is where the new node belongs.
        TNode *node = malloc(sizeof *node);
        // Allocate one tree node.
        if (node == NULL) return NULL;
        // Propagate allocation failure in this compact example.
        node->key = key;
        // Store the search key.
        node->left = NULL;
        // Initialize the left child as empty.
        node->right = NULL;
        // Initialize the right child as empty.
        return node;
```

```
        // Return the new subtree root to the caller.
    }
    if (key < root->key) root->left = insert(root->left, key);
    // Recurse into the smaller-key subtree.
    else if (key > root->key) root->right = insert(root->right, key);
    // Recurse into the larger-key subtree.
    return root;
    // Return the unchanged current root after linking the child.
}

void inorder(const TNode *root) {
// Print keys in sorted order for a valid BST.
    if (root == NULL) return;
    // Stop recursion at an empty child.
    inorder(root->left);
    // Visit all smaller keys first.
    printf("%d ", root->key);
    // Process the current key.
    inorder(root->right);
    // Visit all larger keys last.
}

int main(void) {
// Start the BST demonstration.
    TNode *root = NULL;
```

```
    // Begin with an empty tree.
    root = insert(root, 20);
    // Insert the first key as root.
    root = insert(root, 10);
    // Insert a smaller key into the left subtree.
    root = insert(root, 30);
    // Insert a larger key into the right subtree.
    inorder(root);
    // Validate ordering by printing an inorder traversal.
    printf("\n");
    // Finish the output line.
    return 0;
    // Finish successfully; production code should free nodes.
}
```

Interview revision: Always distinguish tree height, balanced average behavior and worst-case degeneration.

Session 28: Heap and Priority-Based Retrieval

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Heap	A complete binary tree usually stored in an array that maintains a heap-order rule.	Retrieves minimum or maximum priority efficiently.	Process highest-priority restock requests.	Confusing heap order with fully sorted order leads to wrong expectations.	Check parent-child invariant at every index.
Min-heap / max-heap	Min-heap keeps smallest at root; max-heap keeps largest at root.	Lets the root represent highest priority under chosen convention.	Urgency score can use max-heap; lowest cost can use min-heap.	Using inconsistent comparison reverses priority.	Insert known priorities and verify root after each operation.
Heapify	Restores heap order by moving an element up or down.	Makes push/pop logarithmic.	After adding or removing a request, restore priority order.	Wrong child index or stopping condition breaks invariant.	Validate all parent-child relationships after operation.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Priority queue	Abstract behavior: insert items and remove highest/lowest priority.	Heap is a standard implementation strategy.	Schedule urgent requests before normal ones.	Assuming FIFO among equal priority without defining tie policy.	Define tie behavior and test duplicates.

Hands-on practice: Implement max-heap push using an array.

Session 28 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.

#define HCAP 8
// Define the bounded heap capacity.
typedef struct { int data[HCAP]; int size; } Heap;
// Store heap array and logical size together.

int heap_push(Heap *h, int value) {
// Insert one value while preserving max-heap order.
    if (h == NULL || h->size >= HCAP) return 0;
    // Reject invalid heap or overflow.
    int i = h->size;
    // Start at the next free array position.
    h->data[i] = value;
    // Place the new value temporarily at the end.
    h->size++;
    // Increase logical size after storing the new element.
    while (i > 0) {
    // Bubble upward until the root or valid order is reached.
        int parent = (i - 1) / 2;
        // Compute the parent index for a zero-based heap.
```

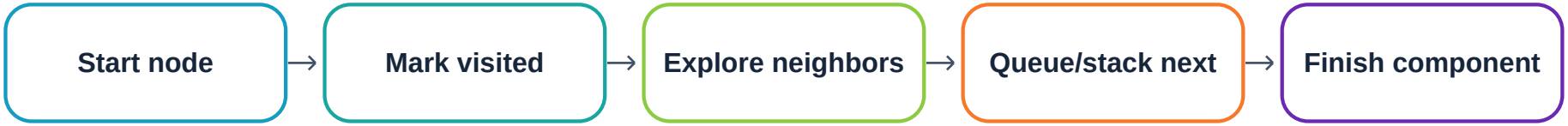
```
        if (h->data[parent] >= h->data[i]) break;
        // Stop when parent is already at least as large.
        int temp = h->data[parent];
        // Save the parent before swapping.
        h->data[parent] = h->data[i];
        // Move the larger child value upward.
        h->data[i] = temp;
        // Move the old parent downward.
        i = parent;
        // Continue checking from the value's new position.
    }
    return 1;
    // Report successful insertion.
}

int main(void) {
// Start the heap test.
    Heap h = {{0}, 0};
    // Initialize an empty heap.
    heap_push(&h, 3);
    // Insert a low priority.
    heap_push(&h, 9);
    // Insert a higher priority that should rise to root.
    printf("max=%d\n", h.data[0]);
    // Verify that root holds the maximum value.
```

```
    return 0;
    // Finish successfully.
}
```

Interview revision: State heap push/pop O(log n), root peek O(1), and why the underlying array is not globally sorted.

Session 29: Graphs, Representations, BFS and DFS

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.



Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Graph	A set of vertices/nodes connected by edges.	Models general relationships that are not strictly hierarchical.	Can model routes between warehouses or dependencies between tasks.	Incorrect visited handling can revisit forever in cyclic graphs.	Test disconnected components, cycles and isolated nodes.
Adjacency matrix/list	Matrix uses VxV cells; adjacency list stores neighbors.	Representation changes memory and traversal cost.	Sparse warehouse routes usually favor adjacency lists.	Matrix wastes space for sparse graphs; list needs pointer/index care.	Compare space against V and E: matrix O(V^2), list O(V+E).

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
BFS	Breadth-First Search explores level by level using a queue.	Finds shortest path in number of edges for unweighted graphs.	Minimum-hop route between sites.	Marking visited too late can enqueue duplicates.	Verify distance layers and visited count.
DFS	Depth-First Search explores deeply before backtracking using recursion/stack.	Useful for reachability, components and ordering patterns.	Check whether all sites are reachable.	Deep recursion can overflow; cycles need visited set.	Trace entry/exit order and visit every reachable node once.

Hands-on practice: Implement BFS on a small adjacency matrix.

Session 29 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.

#define V 4
// Fix a small vertex count for the learning graph.

int main(void) {
// Start the BFS demonstration.
    int graph[V][V] = {{0,1,1,0},{1,0,0,1},{1,0,0,1},{0,1,1,0}};
    // Store an undirected graph as an adjacency matrix.
    int queue[V] = {0};
    // Reserve enough queue space for each vertex once.
    int visited[V] = {0};
    // Track whether each vertex has already been discovered.
    int front = 0;
    // Point to the next queue element to process.
    int rear = 0;
    // Point to the next free queue slot.
    queue[rear++] = 0;
    // Enqueue the starting vertex.
    visited[0] = 1;
    // Mark it visited when enqueued to avoid duplicates.
```

```
    while (front < rear) {
    // Continue while the queue contains work.
        int u = queue[front++];
        // Dequeue the oldest discovered vertex.
        printf("%d ", u);
        // Process the vertex by printing it.
        for (int v = 0; v < V; ++v) {
        // Inspect every possible neighbor in the matrix row.
            if (graph[u][v] && !visited[v]) {
        // Select an adjacent vertex not previously discovered.
                visited[v] = 1;
        // Mark it immediately to prevent duplicate enqueues.
                queue[rear++] = v;
        // Enqueue it for later level-order processing.
            }
        }
    }
    printf("\n");
    // End the traversal output line.
    return 0;
    // Finish successfully.
}
```

Interview revision: For BFS/DFS, always state representation and complexity in terms of V (vertices) and E (edges).

Session 30: Trie: Prefix Search

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Trie	A tree where edges represent characters and paths represent prefixes/words.	Makes prefix lookup depend on word length rather than number of stored words.	Autocomplete/search for product codes or contact names in the advanced project.	Large alphabet arrays can consume significant memory.	Test exact word, prefix-only, absent word and shared prefixes.
Prefix	Beginning sequence of a word.	Different words can share the same trie path.	“key” can lead to Keyboard/Keypad-like product names after normalization.	Confusing prefix existence with full-word existence returns false positives.	Store an end-of-word marker and test both conditions.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Child links	Each node points to next-character nodes.	Represents branching characters.	Different product-name letters branch from shared prefix.	Out-of-range character indexing can corrupt memory.	Normalize/validate character set before mapping to index.
End marker	Boolean/integer flag that a path is a complete stored word.	Separates a full key from a mere prefix.	Recognizes exact product code after traversing characters.	Without it, every prefix may be treated as a stored word.	Insert “car” and “cart”; verify both exact and prefix cases.

Hands-on practice: Implement a lowercase a-z trie insertion skeleton.

Session 30 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.
#include <stdlib.h>                                     // Provide calloc.

#define ALPHA 26
// Support lowercase English letters a through z.
typedef struct TrieNode { struct TrieNode *child[ALPHA]; int is_word; } TrieNode;
// Store child links and exact-word marker.

TrieNode *new_node(void) {
// Create a zero-initialized trie node.
    return calloc(1, sizeof(TrieNode));
    // Zero all child pointers and is_word in one checked-at-caller
    // allocation.
}

int insert(TrieNode *root, const char *word) {
// Add a validated lowercase word to the trie.
    if (root == NULL || word == NULL) return 0;
    // Reject missing inputs before traversal.
    TrieNode *p = root;
    // Begin at the trie root.
```

```
    for (size_t i = 0; word[i] != '\0'; ++i) {
    // Process characters until the string terminator.
        if (word[i] < 'a' || word[i] > 'z') return 0;
        // Enforce the supported alphabet before indexing.
        int index = word[i] - 'a';
        // Map a lowercase character into 0..25.
        if (p->child[index] == NULL) {
        // Detect a path segment not yet created.
            p->child[index] = new_node();
        // Allocate the missing next node.
            if (p->child[index] == NULL) return 0;
        // Stop safely if allocation failed.
        }
        p = p->child[index];
        // Advance along the character path.
    }
    p->is_word = 1;
    // Mark the final node as a complete stored word.
    return 1;
    // Report successful insertion.
}

int main(void) {
// Start a tiny trie demonstration.
    TrieNode *root = new_node();
```

```
    // Create the empty root node.
    printf("insert=%d\n", insert(root, "item"));
    // Insert a lowercase word and print success status.
    return 0;
    // Finish the compact example; production code should free the trie.
}
```

Interview revision: Explain that trie lookup is O(L) for word length L, while memory cost depends heavily on node representation and alphabet size.

Session 31: Searching, Sorting and Complexity Analysis

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Big O	Asymptotic notation describing how time or space grows with input size.	Lets candidates compare solutions against constraints.	Choose between scanning thousands of records or binary-searching sorted IDs.	Ignoring constants entirely or claiming Big O from one timing run is misleading.	Count dominant operations and test growth as n increases.
Best/average/worst case	Different input arrangements can produce different work.	Provides honest performance discussion.	Linear search best case finds first item; worst case scans all.	Quoting only best case hides risk.	State the case being analyzed and its assumptions.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Binary search	Repeatedly halves a sorted search range.	Reduces lookup to O(log n) when random access and sorted order are available.	Search sorted employee IDs.	Using it on unsorted data returns unreliable results; bad mid updates can loop forever.	Verify sortedness and test absent targets around boundaries.
Sorting	Arranges data by a key.	Supports reporting, binary search and greedy algorithms.	Sort employees by ID/salary or products by quantity.	Unstable/incorrect comparator logic can misorder equal keys.	Check sorted invariant and compare element multiset before/after.

Hands-on practice: Implement iterative binary search over sorted integer IDs.

Session 31 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.

int binary_search(const int a[], int n, int target) {
// Search a sorted ascending array for one target.
    int left = 0;
    // Initialize the inclusive lower search bound.
    int right = n - 1;
    // Initialize the inclusive upper search bound.
    while (left <= right) {
    // Continue while a valid search interval remains.
        int mid = left + (right - left) / 2;
        // Compute midpoint without adding two potentially large bounds.
        if (a[mid] == target) return mid;
        // Return the matching index immediately.
        if (a[mid] < target) left = mid + 1;
        // Discard the lower half including the proven-too-small
        // midpoint.
        else right = mid - 1;
        // Discard the upper half including the proven-too-large
        // midpoint.
    }
```

```
    return -1;
    // Report absence when the interval becomes empty.
}

int main(void) {                                    // Start the test.
    int ids[] = {10, 20, 30, 40, 50};
    // Provide sorted input as required by binary search.
    printf("index=%d\n", binary_search(ids, 5, 40));
    // Verify a target near the upper end.
    return 0;
    // Finish successfully.
}
```

Interview revision: Say the precondition first: sorted ascending data. Then explain O(log n) time and O(1) extra space for the iterative version.

Session 32: Two Pointers and Sliding Window

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Two pointers	Uses two indexes that move according to an invariant instead of nested scanning.	Often turns O(n^2) pair-style work into O(n).	Find pair sum in sorted item values or remove duplicates from sorted IDs.	Moving the wrong pointer can skip valid solutions.	State invariant and justify each pointer movement.
Sliding window	Maintains a contiguous range while it moves over data.	Avoids recomputing overlapping ranges.	Find maximum total demand over any K consecutive days.	Forgetting to remove the outgoing element gives wrong sums.	Compare window sum with brute force on small input.
Invariant	A condition kept true throughout an algorithm.	Makes optimized code explainable and provable.	Window sum equals elements from left..right; two-pointer search uses sorted order.	Optimization without a valid invariant can be fast but wrong.	Write invariant in one sentence and test it after each update.

Hands-on practice: Compute maximum sum of any fixed-size window in O(n).

Session 32 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.

int max_window_sum(const int a[], int n, int k) {
// Return the largest sum among all contiguous windows of size k.
    if (a == NULL || k <= 0 || k > n) return 0;
    // Reject impossible window requests in this learning version.
    int window = 0;
    // Initialize the first-window accumulator.
    for (int i = 0; i < k; ++i) window += a[i];
    // Build the sum of the first k elements once.
    int best = window;
    // Use the first valid window as the initial best.
    for (int right = k; right < n; ++right) {
    // Move the window one element right at a time.
        window += a[right];
        // Add the new element entering from the right.
        window -= a[right - k];
        // Remove the old element leaving from the left.
        if (window > best) best = window;
        // Preserve the largest window sum observed so far.
    }
```

```
    return best;
    // Return the final optimum after all windows are evaluated.
}

int main(void) {                                        // Start the test.
    int demand[] = {2, 1, 5, 1, 3, 2};
    // Create a small sequence with hand-checkable windows.
    printf("max=%d\n", max_window_sum(demand, 6, 3));
    // Validate the O(n) sliding-window implementation.
    return 0;
    // Finish successfully.
}
```

Interview revision: Contrast brute-force O(n*k) recomputation with O(n) window maintenance and state the maintained invariant.

Session 33: Recursion Patterns and Backtracking

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Backtracking	Tries a choice, explores consequences, then undoes the choice to try alternatives.	Solves combinatorial search spaces such as subsets and permutations.	Mostly interview practice in this curriculum; can generate combinations of selected products for a scenario.	Forgetting to undo state contaminates later branches.	Check that every valid state is generated once under the chosen rule.
Choice / explore / undo	The three-step mental model of backtracking.	Keeps recursive search structured.	Choose include/exclude for each candidate item.	Global mutable state can make undo mistakes hard to see.	Trace recursion tree for n=3 and compare expected 2^n subsets.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Pruning	Stops exploring branches that cannot lead to a valid solution.	Can dramatically reduce search work.	Skip combinations exceeding a fixed budget/limit.	Incorrect pruning can remove valid answers.	Prove pruning condition and compare with unpruned small cases.

Hands-on practice: Generate all binary include/exclude choices for three items.

Session 33 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.

void generate(int index, int n, int chosen[]) {
// Explore all include/exclude decisions from one position.
    if (index == n) {
    // Stop when every position has received a decision.
        for (int i = 0; i < n; ++i) printf("%d", chosen[i]);
        // Print one complete generated state.
        printf("\n");
        // Separate this state from the next one.
        return;
        // Return to the previous choice point to explore alternatives.
    }
    chosen[index] = 0;
    // Choose to exclude the current item.
    generate(index + 1, n, chosen);
    // Explore all later decisions under exclusion.
    chosen[index] = 1;
    // Change the current decision to inclusion.
    generate(index + 1, n, chosen);
    // Explore all later decisions under inclusion.
```

```
}

int main(void) {
// Start the backtracking demonstration.
    int chosen[3] = {0};
    // Reserve mutable decision state for three positions.
    generate(0, 3, chosen);
    // Begin recursion from the first position.
    return 0;
    // Finish after every state is generated.
}
```

Interview revision: For backtracking, be able to draw the recursion tree and state branching factor, depth and worst-case search size.

Session 34: Greedy Thinking and Bit Manipulation

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Greedy algorithm	Makes the best local choice under a rule that can be shown to lead to a global optimum.	Builds optimization reasoning beyond coding syntax.	Can choose compatible tasks/meetings or prioritize non-overlapping work.	A locally attractive choice may not be globally optimal if the greedy-choice property does not hold.	Provide proof idea or counterexample testing.
Bit mask	An integer whose individual bits encode independent yes/no flags.	Compactly stores sets of small boolean options.	Permissions or record status flags.	Wrong shifts/masks can toggle the wrong flag.	Test set, clear and check operations on one bit at a time.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Shift operators	<< moves bits left; >> moves bits right under integer rules.	Builds masks and handles powers of two.	1u << k creates a flag at position k.	Shifting by invalid widths or signed values can be problematic.	Use unsigned values and validate bit positions.
XOR	^ is exclusive OR; bit is 1 when operands differ.	Common interview tool for toggling and finding unpaired values.	Mostly interview-grade reasoning in this curriculum.	Confusing XOR with exponentiation gives wrong logic.	Use a small truth table and known pairs.

Hands-on practice: Use bit masks to set, clear and test an “active” flag.

Session 34 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>                                      // Provide printf.

int main(void) {
// Start the bit-mask example.
    unsigned int status = 0u;
    // Begin with all status flags cleared.
    const unsigned int ACTIVE = 1u << 0;
    // Define bit 0 as the active flag.
    const unsigned int VERIFIED = 1u << 1;
    // Define bit 1 as the verified flag.
    status |= ACTIVE;
    // Set the active bit without changing other bits.
    status |= VERIFIED;
    // Set the verified bit as well.
    printf("active=%d\n", (status & ACTIVE) != 0u);
    // Isolate and test the active bit.
    status &= ~ACTIVE;
    // Clear only the active bit while preserving others.
    printf("active=%d verified=%d\n",
    // Display both independent states after the update.
           (status & ACTIVE) != 0u, (status & VERIFIED) != 0u);
```

```
        // Convert masked results into readable true/false integers.
    return 0;
    // Finish successfully.
}
```

Interview revision: For greedy problems, do not claim correctness from examples alone; explain the property that makes the local choice safe.

Session 35: Dynamic Programming and Core Graph Algorithms

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
DP	Dynamic Programming stores answers to repeated subproblems.	Turns many exponential recurrences into polynomial algorithms.	Mostly interview preparation here; useful for optimization problems such as knapsack-like resource selection.	Wrong state/transition/base case makes a consistently wrong table.	Define state, transition and base cases before coding; compare with brute force on small input.
Memoization / tabulation	Memoization caches recursive results; tabulation fills a table iteratively.	Two standard DP implementation styles.	Choose based on reachable states and stack concerns.	Mixing state meaning across indexes gives subtle errors.	Label each dimension and inspect tiny tables manually.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Shortest path	Finds a minimum-cost route under stated edge-weight assumptions.	Important graph optimization pattern.	Choose the least-cost route between facilities.	Using BFS on weighted graphs or Dijkstra with negative edges is wrong.	State weight assumptions and verify distances on a hand graph.
Topological sort	Orders vertices of a Directed Acyclic Graph (DAG) so every directed edge goes forward.	Models prerequisite/dependency scheduling.	Order dependent processing tasks.	Cycles mean no valid topological order.	Verify every edge u->v has position(u) < position(v); detect cycles.

Hands-on practice: Implement bottom-up Fibonacci as a minimal DP table example.

Session 35 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

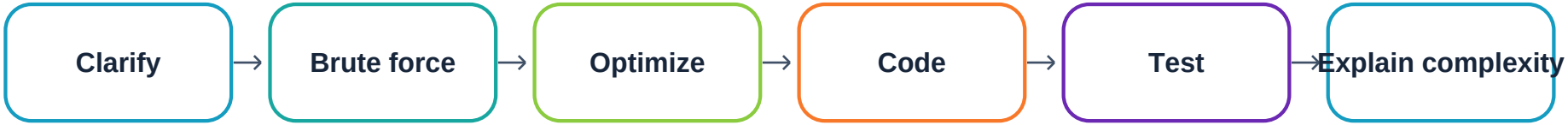
```
#include <stdio.h>                                      // Provide printf.

unsigned long long fib(unsigned int n) {
// Compute Fibonacci using iterative dynamic programming state.
    if (n == 0u) return 0u;
    // Handle the first base case directly.
    if (n == 1u) return 1u;
    // Handle the second base case directly.
    unsigned long long prev2 = 0u;
    // Store F(0), the older subproblem result.
    unsigned long long prev1 = 1u;
    // Store F(1), the most recent subproblem result.
    for (unsigned int i = 2u; i <= n; ++i) {
    // Build answers from the smallest missing state up to n.
        unsigned long long current = prev1 + prev2;
        // Apply the recurrence F(i)=F(i-1)+F(i-2).
        prev2 = prev1;
        // Shift the state window forward by one index.
        prev1 = current;
        // Store the newly computed answer as the latest state.
    }
```

```
    return prev1;
    // Return F(n) after the final iteration.
}

int main(void) {
// Start the DP test.
    printf("F(10)=%llu\n", fib(10));
    // Verify against the known value 55.
    return 0;
    // Finish successfully.
}
```

Interview revision: For DP answers, say: state meaning, recurrence/transition, base case, evaluation order, time complexity and space complexity.

Session 36: Capstone Integration, Timed Coding and Interview Readiness

Session goal: build one interview-ready layer of C knowledge and connect it to the C Inventory & Employee Records capstone.



Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Project integration	Combining syntax, logic, memory, structures, files and DSA into one working program.	Demonstrates skill beyond isolated questions.	Deliver the curriculum-aligned C Inventory & Employee Records capstone, optionally with linked-list contact book/inventory features.	A project that runs only on happy-path input is not inte rview-ready.	Use the curriculum checklist: working demo, readable code, file/data handling, error handling, tests and clear explanation.

Concept	Simple meaning / abbreviation	Why it matters	Project use	Risk if weak	Validate / measure
Timed test	Solving within a fixed time while communicating approach and constraints.	Coding rounds measure reasoning under limited time.	Practice mixed search, array, string, stack/queue and graph problems.	Rushing into code creates rework.	Track correctness, completion time and post-test defect count.
Interview answer flow	Clarify input -> brute force -> optimize -> code -> test cases.	Shows structured thinking, not just memorized code.	Explain project and DSA solutions consistently.	Silent coding hides reasoning and missed assumptions.	Use an answer checklist and score communication separately.
Readiness score	Curriculum rubric: syntax 15%, logic 20%, DSA 20%, algorithms 20%, debugging 10%, readability 5%, project 10%.	Provides measurable preparation status.	Mentor scores final session and sets revision priorities.	A single coding score can hide weak debugging or explanation skills.	Use weighted rubric and evidence from tests/project/mock.

Hands-on practice: Final deliverable: working C capstone + one timed mixed test + one mock explanation of approach, complexity, edge cases and code quality.

Session 36 - Professional C Implementation

The code below is intentionally commented statement by statement so a beginner can explain both *what* each line does and *why* it exists.

```
#include <stdio.h>
// Provide console I/O for the capstone shell.

void print_menu(void) {
// Isolate presentation logic from business operations.
    printf("1 Add product\n");
    // Show the add-record command.
    printf("2 Search product\n");
    // Show the search command.
    printf("3 Save and exit\n");
    // Show the persistence/termination command.
}

int main(void) {
// Start the integrated application shell.
    int choice = 0;
    // Store the current validated menu choice.
    do {
    // Display the menu at least once and repeat until exit.
        print_menu();
        // Reuse a focused function to keep main readable.
```

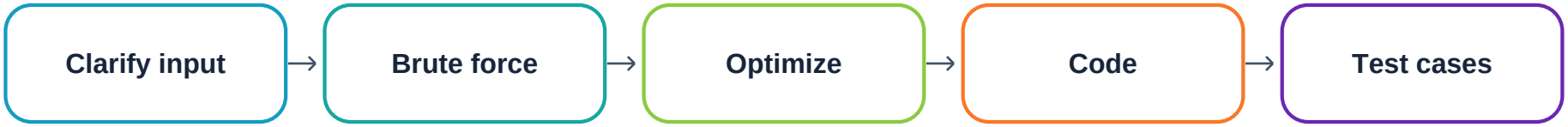
```
        if (scanf("%d", &choice) != 1) {
        // Validate external input before using it.
            printf("Invalid input\n");
        // Give explicit feedback on bad input.
            return 1;
        // Stop this compact shell rather than continuing with invalid
        // state.
        }
        switch (choice) {
        // Dispatch the selected application action.
            case 1: printf("Add flow -> validate -> store\n"); break;
        // Represent the add operation pipeline.
            case 2: printf("Search flow -> find -> display\n"); break;
        // Represent the search operation pipeline.
            case 3: printf("Save -> close files -> free memory\n"); break;
        // Represent controlled cleanup before exit.
            default: printf("Unknown choice\n");
        // Handle unsupported choices safely.
        }
    } while (choice != 3);
    // Continue until the explicit exit command is chosen.
    return 0;
    // Report successful program termination.
}
```

Interview revision: Final mock: explain one array/string problem, one pointer/memory question, one DSA implementation and the capstone architecture.

Final Revision Checklist

Checklist item	Ready when the candidate can...
Syntax	Write basic C programs without copying.
Dry run	Trace variables, loops, recursion and pointer changes on paper.
Debugging	Find off-by-one, invalid input and memory-related errors.
DSA	Choose array, hash table, stack, queue, linked list, tree, heap, graph or trie based on access/update needs.
Algorithms	Explain brute force, optimized approach, Big O, assumptions and edge cases.
Project	Demo the C Inventory & Employee Records capstone with file/data handling and tests.
Interview communication	Clarify input, explain approach, code modularly and test before declaring completion.

Readiness interpretation: below 50 = foundation rebuild; 50-70 = structured DSA/timed practice needed; 70-85 = interview-ready for many service-based roles with mock practice; 85+ = strong readiness for tougher coding rounds and advanced interviews.


Clarify input
Brute force
Optimize
Code
Test cases